Quantum Game Engine + Vanilla Quake Port

Vision

Build a **Quantum Game Engine** that provides genuine quantum computation backends, then port **vanilla Quake** to run on it with quantum-accelerated subsystems.

Goals:

- Run actual Quake gameplay (not a toy demo)
- Use REAL quantum computation (not classically-simulable tricks like Quandoom)
- Create a reusable Quantum Game Engine for future projects
- Produce publishable research on quantum game computation

Architecture Overview

```
┌──────────────────────────────────────────────────────────┐
│                      QUANTUM QUAKE                       │
│     (Vanilla Quake logic → Quantum Rendering Engine)     │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│                QUANTUM GAME ENGINE (QGE)                 │
│   ┌──────────────────────────────────────────────────┐   │
│   │               QUANTUM RENDERER (Core)            │   │
│   │   Game State → Quantum Amplitude → Pixel Output  │   │
│   └──────────────────────────────────────────────────┘   │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐               │
│   │ Quantum  │  │ Quantum  │  │ Quantum  │               │
│   │   RNG    │  │    AI    │  │ Physics  │               │
│   └──────────┘  └──────────┘  └──────────┘               │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│                        MOONLAB                           │
│      (Quantum simulator: state vectors, gates, GPU)      │
└──────────────────────────────────────────────────────────┘
```

CRITICAL: Quantum-Native Rendering

The visuals MUST be rendered by quantum computation. This is the centerpiece.

Core Principle: Probability Distribution = Image

```
|ψ⟩ = Σ amplitude(x, y, depth, color) |x, y, depth, color⟩
               ↓
    Probability |amplitude|² at each (x,y)
               ↓
    Pixel brightness at screen position (x,y)
```

The quantum state's probability distribution over spatial registers IS the rendered frame.

Qubit Layout for Rendering (28 qubits)

Register	Qubits	Purpose
Screen X	6	64 horizontal positions
Screen Y	6	64 vertical positions
Depth	4	16 depth layers (z-buffer)
Color/Material	4	16 colors/textures
Object ID	4	16 game objects
View State	4	Camera angle, effects
Total	**28**	64×64 quantum-rendered frame

Quantum Rendering Pipeline

```
1. INITIALIZE: |ψ⟩ = |0⟩⊗28
               ↓
2. SUPERPOSITION: Apply H gates to screen registers
   |ψ⟩ = (1/64) Σ_{x,y} |x,y,0,0,0,0⟩
               ↓
3. SCENE ENCODING: Apply controlled gates based on game state
   - Walls: Phase flip positions that hit walls
   - Objects: Amplitude at object screen positions
   - Depth: Controlled rotations for z-ordering
               ↓
4. INTERFERENCE: Quantum interference creates the image
   - Hidden surfaces: Destructive interference
   - Visible surfaces: Constructive interference
               ↓
5. EXTRACT IMAGE:
   Option A: Measure many times, histogram = image (slow)
   Option B: GPU-parallel probability extraction (fast)
   Option C: Amplitude estimation (fewer measurements)
               ↓
6. DISPLAY: 64×64 quantum frame → upscale → screen
```

Key Rendering Operations

Wall Occlusion (Z-Buffer via Amplitude)

```
// For each wall at screen position (wall_x, wall_y) at depth wall_z:
// Apply phase to hide things behind it

void quantum_render_wall(quantum_state_t* state,
                         int wall_x, int wall_y, int wall_z) {
    // Mark this (x,y,z) position
    uint64_t wall_state = encode_xyz(wall_x, wall_y, wall_z);

    // Apply amplitude to wall position (makes it visible)
    grover_oracle(state, wall_state);

    // Destructive interference for positions behind wall (z > wall_z)
    for (int z = wall_z + 1; z < 16; z++) {
        uint64_t behind_state = encode_xyz(wall_x, wall_y, z);
        gate_phase_flip(state, behind_state);  // Cancel amplitude
    }
}
```

Sprite Rendering

```
// Render a sprite (enemy, item) at position with color
void quantum_render_sprite(quantum_state_t* state,
                           int sprite_x, int sprite_y, int sprite_z,
                           int color, float opacity) {
    // Create amplitude at sprite's screen position
    uint64_t sprite_state = encode_xyzc(sprite_x, sprite_y, sprite_z, color);

    // Apply rotation to increase amplitude (Grover-like amplification)
    double angle = opacity * M_PI / 2;  // opacity -> rotation angle
    gate_ry(state, get_amplitude_qubit(sprite_x, sprite_y), angle);
}
```

Ray Casting via Superposition

```c
// Cast rays in superposition, let interference find visible surfaces
void quantum_raycast(quantum_state_t* state, vec3_t eye, vec3_t forward) {
    // Put all screen positions in superposition
    for (int q = 0; q < 12; q++) {  // X and Y registers
        gate_hadamard(state, q);
    }

    // For each screen pixel (now in superposition), trace ray
    // This happens "simultaneously" due to quantum parallelism

    // Apply controlled gates based on ray-surface intersections
    // (Pre-computed or encoded as oracle)
    quantum_apply_scene_oracle(state, eye, forward);

    // Interference automatically handles occlusion:
    // - Rays that hit nothing: low amplitude
    // - Rays that hit surfaces: high amplitude at correct depth
}
```

Frame Extraction Methods

Method A: Repeated Measurement (Accurate, Slow)

```c
void extract_frame_sampling(quantum_state_t* state, uint8_t* framebuffer,
                            int samples_per_pixel) {
    for (int s = 0; s < samples_per_pixel * 64 * 64; s++) {
        // Clone state (measurement destroys it)
        quantum_state_t* clone = quantum_state_clone(state);

        // Measure position registers
        int x = measure_register(clone, 0, 6);   // bits 0-5
        int y = measure_register(clone, 6, 6);   // bits 6-11
        int color = measure_register(clone, 16, 4);  // bits 16-19

        // Accumulate in framebuffer
        framebuffer[y * 64 + x] += color_to_brightness(color);

        quantum_state_destroy(clone);
    }
}
```

Method B: Direct Probability Extraction (Fast, GPU)

```
void extract_frame_gpu(metal_compute_ctx_t* metal,
                       metal_buffer_t* quantum_state,
                       metal_buffer_t* framebuffer) {
    // Custom Metal kernel: marginalize over non-position qubits
    // Sum |amplitude|² for each (x,y) over all other registers

    metal_dispatch_kernel(metal, "marginalize_to_image",
                          quantum_state,
                          framebuffer,
                          64, 64);  // Output dimensions
}
```

Method C: Quantum Amplitude Estimation (Fewest Shots)

```
void extract_frame_qae(quantum_state_t* state, uint8_t* framebuffer) {
    for (int x = 0; x < 64; x++) {
        for (int y = 0; y < 64; y++) {
            // Use amplitude estimation to get probability at (x,y)
            // Requires O(1/ε) operations for precision ε
            float prob = quantum_amplitude_estimation(state, x, y, 0.01);
            framebuffer[y * 64 + x] = (uint8_t)(prob * 255);
        }
    }
}
```

Resolution Strategy

Native: 64×64 quantum rendered (4096 pixels, 12 position qubits)

Upscaled: 64×64 → 320×200 or 640×480 via classical upscaling

Hybrid: Quantum provides depth/visibility, classical adds texture detail

ALTERNATIVE: Tensor Network Diffusion Renderer

This is likely the superior approach for achieving high-quality visuals while staying "quantum-native."

Core Idea: Diffusion Model on MPS/MPO

```
┌──────────────────────────────────────────────────────────────┐
│                    RENDERING PIPELINE                        │
│                                                              │
│  Game State      Condition       Diffusion         Frame     │
│  (28 qubits) ──→ Encoding  ──→   Model    ──→   (128×128)  │
│                    │              on MPS                     │
│                    │                │                        │
│                    └────────────────┘                        │
│                    Tensor Network Operations                 │
│                    (Native Moonlab MPS/MPO)                  │
└──────────────────────────────────────────────────────────────┘
```

Why Neural Networks ARE Tensor Networks

A feedforward layer `y = Wx + b` is a tensor contraction:

- Input `x` : vector (rank-1 tensor)
- Weights `W` : matrix (rank-2 tensor)
- Output `y` : tensor contraction

An MPS naturally represents:

- Network weights as bond tensors
- Activations as site tensors
- Forward pass as sequential contraction

Diffusion Model Architecture (Compact)

Target: ~1-5M parameters (fits in MPS with reasonable bond dimension)

```
Tiny U-Net for 128×128:
├── Encoder: 3 downsample blocks
├── Middle: 2 residual blocks + attention
├── Decoder: 3 upsample blocks
└── Conditioning: game state embedding

MPS Representation:
- Bond dimension: 256-512
- ~100-200 MPS sites
- Memory: ~50-200 MB
```

Training Strategy

Phase 1: Collect Training Data

```python
# Run vanilla Quake, capture frame pairs
for episode in range(10000):
    game_state = get_game_state()  # Position, enemies, items
    frame = render_frame_classical()  # Vanilla software renderer
    save_pair(game_state, frame)
```

Phase 2: Train Diffusion Model

```python
# Standard diffusion training
model = TinyUNet(condition_dim=64)  # 64 = encoded game state
for batch in dataloader:
    noise = torch.randn_like(batch.frame)
    t = sample_timestep()
    noisy = add_noise(batch.frame, noise, t)
    predicted = model(noisy, t, batch.game_state)
    loss = mse(predicted, noise)
    loss.backward()
```

Phase 3: Convert to MPS

```python
# Convert trained weights to MPS format
mps_model = convert_to_mps(model, bond_dim=512)
export_moonlab_format(mps_model, "quake_diffusion.mps")
```

MPS Inference in Moonlab

```
// Load pre-trained diffusion model as MPS
mps_state_t* diffusion_model = mps_load("quake_diffusion.mps");

// Render a frame
void quantum_diffusion_render(mps_state_t* model,
                              quantum_state_t* game_state,
                              uint8_t* framebuffer) {
    // Extract game state encoding from quantum measurement
    float condition[64];
    extract_game_state_encoding(game_state, condition);

    // Initialize with noise (quantum RNG)
    float latent[128][128];
    quantum_noise_init(latent, 128, 128);

    // Diffusion denoising steps (TDVP-like)
    for (int t = 1000; t > 0; t -= 50) {  // 20 steps
        float timestep_embedding[64];
        encode_timestep(t, timestep_embedding);

        // MPS forward pass: predict noise
        float predicted_noise[128][128];
        mps_forward_pass(model, latent, timestep_embedding,
                         condition, predicted_noise);

        // Denoise step
        denoise_step(latent, predicted_noise, t);
    }

    // Convert to framebuffer
    latent_to_rgb(latent, framebuffer);
}
```

Advantages Over Pure Amplitude Rendering

Aspect	Amplitude Rendering	Diffusion on MPS
Resolution	64×64 max	128×128+ achievable
Visual quality	Probabilistic, noisy	Learned, coherent

Aspect	Amplitude Rendering	Diffusion on MPS
Color depth	16 colors (4 qubits)	Full RGB
Texture detail	None	Learned from training
Frame rate	~10 FPS	~15-30 FPS
Genuinely quantum?	Yes (state vector)	Yes (tensor network)

Hybrid: Best of Both Worlds

```
┌──────────────────────────────────────────────────────────┐
│                    HYBRID RENDERER                        │
│                                                           │
│  ┌──────────────┐     ┌──────────────┐     ┌──────────────┐  │
│  │   Quantum    │     │   Quantum    │     │   Diffusion  │  │
│  │  Game State  │ ──→ │  Low-Res     │ ──→ │   Upscaler   │  │
│  │  (28 qubit)  │     │  (32×32)     │     │    (MPS)     │  │
│  └──────────────┘     └──────────────┘     └──────────────┘  │
│        │                   │                   │            │
│        │                   │                   ▼            │
│        │                   │            ┌──────────────┐    │
│        │                   └──────────→ │  128×128     │    │
│        │                                │   Output     │    │
│        └──────────────────────────────→ └──────────────┘    │
│              Game state conditioning                      │
└──────────────────────────────────────────────────────────┘

Both paths use genuine quantum/tensor network computation!
```

Required Moonlab Extensions

1. **MPS-based Neural Network Primitives**
   - `mps_linear()` - Dense layer as MPS contraction
   - `mps_conv2d()` - Convolution via MPO
   - `mps_attention()` - Self-attention via tensor contraction
2. **Diffusion Utilities**
   - `mps_noise_schedule()` - DDPM/DDIM schedules
   - `mps_denoise_step()` - Single denoising iteration
   - `mps_cfg_guidance()` - Classifier-free guidance

3. **Metal GPU Kernels**
    - MPS contraction on GPU
    - Batched tensor operations
    - Latent-to-RGB conversion

REAL-TIME: Probability Budget Rendering

The NES Sprite Analogy

```
┌──────────────────────────────────────────────────────────┐
│  NES HARDWARE LIMIT          QUANTUM PROBABILITY BUDGET  │
│  ═══════════════════         ═══════════════════════════ │
│                                                          │
│  64 sprites total      ←→    Σ |amplitude|² = 1.0        │
│  8 sprites/scanline    ←→    Probability per pixel row   │
│  Sprite flickering     ←→    Interference / superposition │
│  Priority by order     ←→    Z-depth encoded in amplitude │
└──────────────────────────────────────────────────────────┘
```

Core Principle: Probability IS the Framebuffer

No classical rendering. No intermediate steps.

```
// The quantum state IS the frame
// P(x,y) = brightness at pixel (x,y)

typedef struct {
    quantum_state_t* state;     // 28-qubit state = the "framebuffer"
    metal_buffer_t* gpu_state;  // GPU mirror for fast extraction
    float* probability_cache;   // 64x64 probability grid
} quantum_framebuffer_t;
```

Object Probability Costs

Like NES sprites have a "cost", quantum objects cost probability amplitude:

Object Type	Probability Cost	Screen Coverage
Wall segment	0.02-0.05	2-4 pixels
Enemy sprite	0.05-0.10	4-8 pixels
Player weapon	0.03	2-3 pixels
Projectile	0.01	1 pixel
Pickup item	0.02	2 pixels
Background	~0.50	Remaining pixels

Total budget: 1.0 (enforced by quantum normalization)

Real-Time Frame Pipeline (Target: 30 FPS = 33ms)

```
Frame Timeline (33ms budget):
├── [0-5ms]   Input processing, gate selection
├── [5-15ms]  Apply quantum gates (GPU accelerated)
├── [15-25ms] Extract probabilities (Metal kernel)
├── [25-30ms] Probability → RGB conversion
└── [30-33ms] Display + buffer swap
```

Fast Probability Extraction (The Key to Real-Time)

```
// Metal kernel: Extract 64x64 probability grid in ~10ms
kernel void extract_probabilities(
    device const float2* amplitudes [[buffer(0)]],  // Complex amplitudes
    device float* probabilities [[buffer(1)]],       // 64x64 output
    constant uint& total_states [[buffer(2)]],       // 2^28
    uint2 gid [[thread_position_in_grid]]
) {
    uint x = gid.x;  // 0-63
    uint y = gid.y;  // 0-63

    float prob = 0.0;

    // Marginalize over all non-position qubits
    // Position in bits 0-11 (6 bits X, 6 bits Y)
    uint target_xy = (y << 6) | x;

    // Sum over all states where (x,y) matches
    for (uint i = 0; i < total_states; i++) {
        if ((i & 0xFFF) == target_xy) {
            float2 amp = amplitudes[i];
            prob += amp.x * amp.x + amp.y * amp.y;  // |α|²
        }
    }

    probabilities[y * 64 + x] = prob;
}
```

Optimization: Strided Access Pattern

```
// Even faster: exploit memory layout
// States with same (x,y) are spaced by 2^12 = 4096

kernel void extract_probabilities_fast(
    device const float2* amplitudes [[buffer(0)]],
    device float* probabilities [[buffer(1)]],
    uint2 gid [[thread_position_in_grid]]
) {
    uint xy = (gid.y << 6) | gid.x;
    float prob = 0.0;

    // Stride through all states with this (x,y)
    for (uint offset = 0; offset < (1 << 16); offset++) {  // 2^16 = non-position state
        uint state_idx = (offset << 12) | xy;
        float2 amp = amplitudes[state_idx];
        prob += amp.x * amp.x + amp.y * amp.y;
    }

    probabilities[gid.y * 64 + gid.x] = prob;
}
```

Quantum "Sprite" Operations

Spawn Enemy (Allocate Probability)

```
void quantum_spawn_enemy(quantum_framebuffer_t* fb,
                         int screen_x, int screen_y,
                         int enemy_type, float visibility) {
    // Concentrate probability at enemy position
    // Uses Grover-like amplitude amplification

    uint64_t enemy_state = encode_screen_pos(screen_x, screen_y)
                         | encode_enemy_type(enemy_type);

    // Amplify this state (costs probability from elsewhere)
    grover_amplify(fb->state, enemy_state, visibility);
}
```

Move Enemy (Shift Probability)

```c
void quantum_move_enemy(quantum_framebuffer_t* fb,
                        int old_x, int old_y,
                        int new_x, int new_y) {
    // Swap amplitude between old and new positions
    // No probability created or destroyed

    int old_qubits[12], new_qubits[12];
    encode_position_qubits(old_x, old_y, old_qubits);
    encode_position_qubits(new_x, new_y, new_qubits);

    // Controlled swap: moves amplitude from old to new
    quantum_swap_positions(fb->state, old_qubits, new_qubits);
}
```

Destroy Enemy (Release Probability)

```c
void quantum_destroy_enemy(quantum_framebuffer_t* fb,
                           int screen_x, int screen_y) {
    // Measure the enemy position → collapses that state
    // Released probability redistributes to background

    uint64_t enemy_pos = encode_screen_pos(screen_x, screen_y);

    // Partial measurement: collapse enemy states
    measurement_partial(fb->state, enemy_pos);

    // Re-normalize: probability goes to remaining objects
    quantum_state_normalize(fb->state);
}
```

Real-Time Optimizations

1. **Keep state on GPU permanently** - No CPU↔GPU transfers per frame
2. **Double buffering** - Render frame N+1 while displaying frame N
3. **Incremental updates** - Only apply gates for changed objects
4. **LOD probability** - Distant objects get less probability (naturally fades)
5. **Temporal coherence** - Reuse previous frame's probabilities as hint

Frame Rate Scaling

If we can't hit 30 FPS, gracefully degrade:

Scenario	Resolution	Objects	Target FPS
Full quality	64×64	20	30
Medium	48×48	15	45
Fast	32×32	10	60

Resolution reduced by using fewer position qubits:

- 64×64 = 12 qubits → 16 qubits for game state
- 32×32 = 10 qubits → 18 qubits for game state

The Flicker Effect (Feature, Not Bug!)

Like NES sprite flickering, quantum probability flickering is **authentic**:

- Objects in superposition naturally "shimmer"
- Measurement collapse creates dramatic "solidification"
- Interference patterns create moiré-like visual effects

Lean into it for aesthetic!

Part 1: Quantum Game Engine (QGE)

Module 1: Quantum RNG (`qge_rng`)

Replace Quake's M_Random() with genuine quantum randomness.

Implementation:

- 16-qubit register in superposition (Hadamard on all qubits)
- Measure to collapse to random 16-bit value
- Batch measurements for performance (measure once, use multiple bits)
- Hardware entropy seeding from Moonlab's entropy pool

Qubits: 16 dedicated

API:

```
uint16_t qge_random(void);           // Single quantum random value
void qge_random_batch(uint16_t* out, int count);  // Batched for performance
float qge_random_float(void);        // 0.0-1.0 uniform
```

Module 2: Quantum AI (`qge_ai`)

Enemy decision-making via quantum superposition and measurement.

Implementation:

- Each enemy type gets a quantum decision register (4-8 qubits)
- AI states exist in superposition until decision point
- Measurement collapses to chosen action
- Entangle enemies for coordinated swarm behavior

Qubits: 8 per active enemy (max 3 enemies = 24 qubits simultaneous)

API:

```
typedef enum { AI_IDLE, AI_PATROL, AI_CHASE, AI_ATTACK, AI_FLEE } ai_action_t;
ai_action_t qge_ai_decide(int enemy_id, float aggression, float player_distance);
void qge_ai_entangle(int enemy_a, int enemy_b);  // Link enemy decisions
```

Module 3: Quantum Physics (`qge_physics`)

Particle systems and physics approximation via quantum simulation.

Implementation:

- Encode particle positions in qubit registers
- Use quantum gates for physics evolution
- TDVP-style time evolution for continuous dynamics
- Measure to extract particle positions for rendering

Qubits: 20-28 for particle system

API:

```
void qge_particle_spawn(vec3_t position, vec3_t velocity);
void qge_particle_evolve(float dt);
void qge_particle_render(particle_t* out, int* count);
```

Module 4: Quantum Visibility (`qge_vis`)

Grover-accelerated BSP traversal for visible surface determination.

Implementation:

- Encode BSP node indices in qubit register
- Use Grover oracle to mark "visible from viewpoint" nodes
- Amplitude amplification concentrates probability on visible set
- Measure to sample visible surfaces

Qubits: 16-20 (for BSP with up to 2^20 nodes)
Speedup: Quadratic over classical (√N instead of N)
API:

```
void qge_vis_setup_viewpoint(vec3_t eye, vec3_t forward);
int qge_vis_query_surface(int surface_id);  // Returns visibility probability
void qge_vis_get_visible_set(int* surfaces, int* count, int max_count);
```

Part 2: Vanilla Quake Port

Source Base

- Start from id-Software/Quake GPL release
- Use WinQuake (software renderer) as base - simpler than GLQuake
- Preserve as much vanilla code as possible

Integration Points

1. Replace M_Random() (common.c)

```
// Original:
// static unsigned char rndtable[256] = { ... };
// int M_Random(void) { return rndtable[rndindex++ & 0xff]; }

// Quantum replacement:
int M_Random(void) {
    return qge_random() & 0xff;
}
```

2. Hook Enemy AI (ai.c, monster.c)

```
// In monster think functions, replace:
// if (random() < 0.3) attack();

// With quantum decision:
ai_action_t action = qge_ai_decide(self->enemy_id,
                                   self->aggression,
                                   distance_to_player);
switch (action) {
    case AI_ATTACK: monster_attack(); break;
    case AI_CHASE:  monster_chase();  break;
    // ...
}
```

3. Hook Particle System (r_part.c)

```
// Replace classical particle update with quantum evolution
void R_UpdateParticles(float frametime) {
    qge_particle_evolve(frametime);
    qge_particle_render(r_particles, &r_numparticles);
}
```

4. Hook Visibility (r_bsp.c)

```
// Enhance R_RecursiveWorldNode with quantum visibility hints
void R_RecursiveWorldNode(mnode_t *node) {
    // Query quantum visibility oracle
    if (qge_vis_query_surface(node->surface_id) < 0.01) {
        return;  // Quantum says this is unlikely visible
    }
    // ... rest of vanilla traversal
}
```

Files to Modify in Quake Source

File	Modification
`common.c`	Replace M_Random with qge_rng
`mathlib.c`	Add quantum vector operations
`ai.c`	Hook qge_ai for decision making
`monster.c`	Per-monster quantum state init
`r_part.c`	Quantum particle system
`r_bsp.c`	Quantum visibility queries
`chase.c`	Quantum-enhanced camera
`cl_main.c`	Initialize QGE on client start
`sv_main.c`	Initialize QGE on server start
`Makefile`	Link against Moonlab + QGE

Part 3: Project Structure

```
quantum_quake/
├── Makefile
├── CMakeLists.txt
├── README.md
│
├── qge/                          # Quantum Game Engine
│   ├── qge.h                     # Main header
│   ├── qge_init.c                # Engine initialization
│   ├── qge_rng.c                 # Quantum RNG module
│   ├── qge_rng.h
│   ├── qge_ai.c                  # Quantum AI module
│   ├── qge_ai.h
│   ├── qge_physics.c             # Quantum physics module
│   ├── qge_physics.h
│   ├── qge_vis.c                 # Quantum visibility module
│   ├── qge_vis.h
│   └── qge_moonlab.c             # Moonlab integration layer
│
├── quake/                        # Modified Quake source
│   ├── (vanilla Quake source files)
│   ├── qge_hooks.c               # QGE integration hooks
│   └── qge_hooks.h
│
├── deps/
│   └── moonlab/                  # Git submodule
│
├── assets/                       # Quake shareware data (id1/pak0.pak)
│
└── tests/
    ├── test_qge_rng.c
    ├── test_qge_ai.c
    └── bench_quantum_vs_classical.c
```

Part 4: Implementation Phases

Phase 1: Foundation

- [ ] Set up project structure
- [ ] Clone Quake GPL source
- [ ] Add Moonlab as submodule
- [ ] Build vanilla Quake to verify baseline works
- [ ] Create QGE skeleton with stubs

Phase 2: Quantum RNG

- [ ] Implement qge_rng using Moonlab state vectors
- [ ] Batch measurement optimization
- [ ] Replace M_Random() in Quake
- [ ] Verify game still runs correctly
- [ ] Benchmark quantum vs pseudo-random

Phase 3: Quantum AI

- [ ] Design enemy state encoding
- [ ] Implement qge_ai_decide()
- [ ] Implement enemy entanglement
- [ ] Hook into monster.c think functions
- [ ] Test with single enemy type (grunt)
- [ ] Expand to all enemy types

Phase 4: Quantum Physics

- [ ] Design particle state encoding
- [ ] Implement quantum particle evolution
- [ ] Hook into r_part.c
- [ ] Test with explosion particles
- [ ] Optimize for frame rate

Phase 5: Quantum Visibility

- [ ] Implement Grover oracle for BSP visibility
- [ ] Integrate with r_bsp.c

- [ ] Benchmark speedup on complex maps
- [ ] Tune number of Grover iterations

Phase 6: Polish & Documentation

- [ ] Performance optimization pass
- [ ] Metal GPU acceleration for all modules
- [ ] Write technical documentation
- [ ] Create demo video
- [ ] Prepare for potential paper submission

Technical Constraints

Qubit Budget (28 total for state vector)

Module	Qubits	Notes
RNG	16	Batched, reused
AI (per enemy)	8	Max 3 active = 24
Physics	20	Time-shared with AI
Visibility	16	Time-shared with physics

Strategy: Time-multiplex modules - not all run simultaneously

Performance Targets

Metric	Target
Frame rate	30+ FPS
Quantum ops/frame	<50ms budget
Memory	<8GB total

Platform

- Primary: macOS with Apple Silicon (Metal GPU)

- Secondary: Linux x86-64 (AVX2/AVX-512)

Key Moonlab Integration Points

Critical files from Moonlab:

- `src/quantum/state.h` - State vector management
- `src/quantum/gates.h` - Gate operations
- `src/quantum/measurement.h` - Collapse and sampling
- `src/optimization/gpu_metal.h` - GPU acceleration
- `src/algorithms/grover.h` - For visibility module
- `src/utils/quantum_entropy.h` - Hardware entropy for RNG seeding

Success Criteria

1. **Playable Quake** - Complete E1M1 with quantum subsystems active
2. **Genuine Quantum** - All quantum modules use real superposition/measurement
3. **Measurable Impact** - Demonstrate where quantum provides advantage
4. **Reusable Engine** - QGE usable for other games (Doom port next?)
5. **Publishable** - Novel enough for workshop/conference paper

Final Decisions

- **Target Game:** Quake 1 (1996) - simplest codebase, software renderer, most iconic
- **Rendering Approaches:** Implement ALL THREE, benchmark, keep best:
  i. Pure probability rendering (64×64)
  ii. Diffusion on MPS (128×128)
  iii. Hybrid quantum + diffusion upscaler
- **Scope:** Build the whole thing - QGE + full Quake port
- **Project Location:** Separate repo from Moonlab

Implementation Phases (Concrete)

Phase 1: Project Setup (Week 1)

- [ ] Create `quantum_quake/` repository
- [ ] Add Moonlab as git submodule
- [ ] Clone Quake GPL source from id-Software/Quake
- [ ] Verify vanilla Quake builds and runs
- [ ] Create QGE skeleton with stub APIs

Phase 2: Quantum Framebuffer Core (Weeks 2-3)

- [ ] Implement `quantum_framebuffer_t` structure
- [ ] Create 28-qubit state initialization
- [ ] Implement probability extraction (CPU baseline)
- [ ] Port probability extraction to Metal GPU kernel
- [ ] Benchmark: achieve <20ms extraction time
- [ ] Test: display probability grid as grayscale image

Phase 3: Pure Probability Renderer (Weeks 3-4)

- [ ] Implement quantum wall rendering (amplitude encoding)
- [ ] Implement quantum sprite rendering (Grover amplification)
- [ ] Implement depth buffer via amplitude phases
- [ ] Create probability→RGB color mapping
- [ ] Test: render static scene at 64×64
- [ ] Optimize for 30 FPS target

Phase 4: Diffusion Model Pipeline (Weeks 4-6)

- [ ] Set up training data collection from vanilla Quake
- [ ] Design compact U-Net (~2M parameters)
- [ ] Train diffusion model (game state → frame)
- [ ] Convert trained model to MPS format
- [ ] Implement MPS inference in Moonlab
- [ ] Test: generate frames from game state conditioning

Phase 5: Hybrid Renderer (Week 6)

- [ ] Implement low-res quantum (32×32) path
- [ ] Connect quantum output as diffusion conditioning
- [ ] Implement diffusion upscaling (32→128)
- [ ] Benchmark all three approaches
- [ ] Select best performer for default

Phase 6: Quake Integration (Weeks 7-9)

- [ ] Replace M_Random() with quantum RNG
- [ ] Hook quantum framebuffer into refresh loop
- [ ] Implement view transformation (3D world → 2D quantum frame)
- [ ] Port BSP traversal to quantum visibility
- [ ] Integrate enemy AI with quantum decision module
- [ ] Test: play E1M1 with quantum rendering

Phase 7: Polish & Performance (Weeks 9-10)

- [ ] Profile and optimize hot paths
- [ ] Implement double buffering
- [ ] Add quantum-specific visual effects
- [ ] Sound integration
- [ ] Documentation and README
- [ ] Demo video recording

Key Source Files (Moonlab)

File	Purpose
`src/quantum/state.h`	State vector creation/manipulation
`src/quantum/gates.h`	All quantum gate operations
`src/quantum/measurement.h`	Measurement and collapse
`src/optimization/gpu_metal.h`	Metal GPU acceleration
`src/algorithms/grover.h`	Amplitude amplification

File	Purpose
`src/algorithms/tensor_network/mps.h`	MPS for diffusion model
`src/algorithms/tensor_network/tdvp.h`	Time evolution for diffusion

Key Source Files (Quake)

File	Modification
`WinQuake/common.c`	Replace M_Random()
`WinQuake/r_main.c`	Hook quantum framebuffer
`WinQuake/r_bsp.c`	Quantum visibility integration
`WinQuake/r_draw.c`	Quantum sprite rendering
`WinQuake/ai.c`	Quantum AI decisions
`WinQuake/cl_main.c`	Initialize QGE

Success Metrics

1. **Playable:** Complete E1M1 with quantum rendering at 20+ FPS
2. **Authentic:** All visuals come from quantum state probability
3. **Performant:** Real-time on Apple Silicon (M1+)
4. **Novel:** Publishable as research contribution
5. **Extensible:** QGE reusable for Doom port next